

Refactored Code Smells

Use of primitives and Data clumps

Smell: Every class in the board and entity packages initially used two variables `int x`, `int y` to define its position for the constructor and other methods. The `Position` class existed only as a helper to convert and calculate distances.

Solution (520d858): Parameters accepting `int x`, `int y` are changed to accept a `Position` object instead. This includes creating a new `Position` object as arguments in many locations.

Long method in Antivirus update code

Smell: The antivirus's A* pathfinding code was quite dense, nested in the update code, and at times difficult to understand. The logic itself was quite self-contained, so it was an ideal candidate.

Solution (0404a3f): Extracted the A* pathfinding code into to a new method. Within that method, I found that the functionality of the path tracing code was not immediately clear, so I also extracted and documented that as an additional function (`backtrackUntil`).

Feature envy (inappropriate coupling) of Entity code towards render logic

Smell: The methods `syncToView` and `createRenderNode` in the `Entity` class were only ever utilized by the UI to render the entities on screen. This coupling of game logic classes defining rendering logic was inappropriate as the game loop should not be responsible for informing the UI of the exact sprites to draw. This also meant that changes to the rendering system would *also* require changes to each entity class.

Solution (3f59607, 2e4c8e8): Delete the two methods and create a registry to handle the creation and synchronization of render nodes for specific entity types, purely on the UI side. This made the entity classes fully independent of JavaFX, with render code reading state instead of being sent it.

Long method in Board tick

Smell: The logic to spawn a bonus reward (source code) at a random location is nested inside the `tick()` method of the `Board` class. This logic was independent and self-contained from other instructions in `tick()`.

Solution (4873a72): The spawning logic is extracted into a new method called `spawnSourceCode()`, where `tick()` would call.

Confusing class hierarchy in `Collectable` entities

Smell: The special items (Decryption key, Freeze token) act like other rewards on the board, where the player can collect. These items, however, extended `Entity` directly and implemented their own collision logic that already existed as part of `Collectable`.

Solution ([f926833](#)): These items are changed to extend `Collectable` instead, with a set value of 0 (no score gained/lost). A new abstract method is also created in `Collectable` so that these items can override to execute custom code as the player collects them.

Confusing `TileType` class hierarchy, duplicated code, and feature envy

Smell: `TileType` had 6 subclasses, 5 of which were practically identical in functionality. Each of these classes just overrode a single rendering-related method and implemented it in almost the exact same way. Additionally, the render logic for the tiles should have been in the purview of the `ui` package and not the game logic.

Solution ([fd48fea](#)): Refactored the rendering logic to use a registry, similar to (but not exactly the same as) the entity render refactor, decoupling from the rendering code. `TileType` was made concrete and constructed directly, allowing for the deletion of the 5 useless subclasses. The class `TileTypes` was added to store all the tile singletons so that they're easily accessible in one place.

Duplicated button code on the title screen

Smell: The two buttons on the title screen have the same UI layout, but the code to configure these layouts were duplicated for each button.

Solution ([3ccbc60](#)): A method is created in `TitleMenu` to create buttons, accepting the text, icon, and on-click action as parameters.

Duplicated code, use of unsafe constructs in resource management

Smell: Asset loading code was duplicated in `AudioManager` from `ResourceManager`. The code also faced additional complexity from opening asset streams itself rather than relying on JavaFX to do so when loading. In addition, exceptions that occurred during asset loading were swallowed by broad `try...catch (Exception)` blocks, potentially hiding errors.

Solution ([8b05975](#)): Audio loading code was moved into `ResourceManager`, and asset URIs are now passed instead of open file streams, removing the complexity of having to close them. When images are loaded, they now are checked for an error state before returning, and audio loading errors are caught by the specific `MediaException` instead of all `Exceptions`.